LOCAL MULTI-AGENT CODING SYSTEM

Manager + Specialized 8B Workers on Consumer Hardware

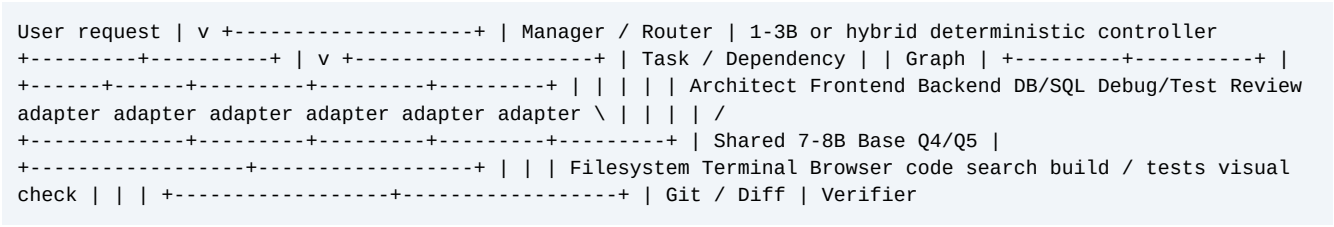
A full implementation blueprint for a locally hosted software-engineering agent that plans, writes, tests, debugs, reviews, and iterates on real web applications.

Core hypothesis

A coordinated collection of small specialist models, sharing one strong 7-8B coding base through lightweight adapters and controlled by a small manager/router, can outperform a single general model on selected software-engineering workflows while remaining deployable on roughly 6-8 GB VRAM.

Component	Target
Manager/router	1-3B quantized model or deterministic state machine + small LLM
Worker model	One 7-8B coding base, Q4/Q5/Q6
Specialists	Architect, frontend, backend, database, debugger/tester, reviewer
Specialization	LoRA/adapter per role rather than six complete 8B checkpoints
Hardware target	6-8 GB VRAM; preferably 32 GB+ system RAM
Primary demo	Autonomously build and repair a Next.js + NestJS + PostgreSQL application
Research question	Can specialist routing + tools beat a single general coding model under a comparable compute/task budget?

1. System Architecture



The key design choice is **shared weights**. Six full 8B models would require tens of gigabytes just for quantized weights. One quantized base plus small role adapters avoids that waste and also makes switching roles much cheaper.

Role	Responsibility
Manager	Decompose goals, track state, choose worker, decide whether to retry/escalate/finish.
Architect	Design modules, interfaces, schema boundaries, dependencies, acceptance criteria.
Frontend	React/Next.js UI, state, API integration, accessibility, browser fixes.
Backend	API, NestJS/Node/Go services, validation, auth, business logic.
Database	PostgreSQL/SQL, Prisma schemas, migrations, queries, indexes.
Debugger/Tester	Interpret failures, create tests, reproduce bugs, repair builds.
Reviewer	Inspect diff for correctness, security, maintainability, missing requirements.

2. Why It Can Fit on 6-8 GB VRAM

The system should **not** keep every worker resident. Only the active base model and required adapter need to be on the GPU. The manager can be CPU-resident, temporarily GPU-resident, or replaced partly by deterministic orchestration.

Memory item	Approximate design assumption
7-8B Q4 weights	~4-5 GB before runtime-specific overhead
KV cache	Variable; controlled through context size, cache precision, and concurrency
Runtime/CUDA buffers	Reserve headroom; exact amount depends on engine
LoRA adapter	Small relative to base weights; role-dependent
Manager	Prefer CPU/RAM or a very small quantized model
System RAM	32 GB recommended so weights, repo index, processes, and offload have room

Memory strategy

- Load one shared 7-8B base and hot-swap role adapters where the runtime supports it.
- Keep the manager on CPU or use deterministic routing for obvious transitions.
- Use sequential agents, not simultaneous six-model inference.
- Limit active context by retrieving only relevant files/symbols.
- Summarize completed subtasks into compact state rather than carrying every token forever.
- Use Q4/Q5 for the deployment target; retain BF16/Q8 only for quality reference and experiments.

Important

Your RTX 4050-class 6 GB target is tighter than 8 GB, but still useful for this research prototype if you use Q4, conservative context, CPU offload, and sequential worker execution. An 8 GB card gives noticeably more breathing room.

3. Autonomous Web-Development Workflow

Example user goal: **Build a production-style task manager with Next.js, NestJS, PostgreSQL, authentication, tests, and Docker.**

```
1. Manager parses objective and acceptance criteria. 2. Architect creates task graph: - repository bootstrap - database schema - backend API - authentication - frontend - tests - integration 3. Manager selects DB worker. 4. DB worker writes Prisma schema + migration. 5. Tool runner validates migration. 6. Backend worker implements endpoints. 7. Build/test verifier runs. 8. Debug worker receives only failure + relevant files. 9. Frontend worker implements pages/components. 10. Browser tool checks rendering and behavior. 11. Reviewer inspects complete git diff. 12. Manager routes findings to appropriate worker. 13. Full test/build suite runs. 14. System returns final diff, test evidence, architecture summary, and unresolved risks.
```

This feedback loop matters more than giving the model a theatrical job title. The compiler, test suite, browser, and git diff provide objective signals that small models can use to correct themselves.

4. Agent Tooling

Minimal tool contract

```
list_files(path) search_code(query, path?) find_symbol(name) read_file(path, start?, end?) write_file(path, content) apply_patch(diff) run_command(command, cwd, timeout) run_build() run_tests() git_diff() git_status() browser_open(url) browser_screenshot() browser_console() browser_click(selector) browser_type(selector, text)
```

Every tool should return structured output with status, stdout/stderr, changed files, duration, and truncation metadata. Never let the model infer success merely because it issued a command.

Sandbox rules

- Execute generated code in disposable containers or VMs.
- Disable network by default; explicitly allow package registries only when needed.
- Limit CPU, memory, process count, disk, command duration, and repository scope.
- Do not expose host secrets, SSH keys, cloud credentials, or arbitrary filesystem paths.
- Require explicit policies for destructive database/filesystem commands.

5. Manager / Router Design

Start with a **hybrid controller**, not a fully learned manager. Deterministic rules make common transitions reliable, while the manager model handles ambiguous planning.

Decision	Preferred mechanism
Build failed	Deterministically route to debugger with build output
Tests failed	Debugger/tester
Schema task	Database specialist
UI task	Frontend specialist
API/business logic	Backend specialist
Large ambiguous feature	Manager/architect planning
All tasks pass	Reviewer, then final verifier
Repeated failure	Manager chooses alternate strategy or escalates context

Manager state

```
project_goal acceptance_criteria[] task_graph[] completed_tasks[] active_task active_worker changed_files[]
test_state build_state known_failures[] architecture_summary token/tool budget retry_count
```

Later, train the manager on successful orchestration traces: task decomposition, worker selection, failure recovery, and stop decisions. Reward final project correctness and penalize unnecessary worker switches/tool calls.

6. Specialist Training Strategy

Use a strong coding base, then train role-specific adapters. Begin with SFT. Add preference optimization only after the data and evaluation pipeline are trustworthy.

Adapter	Training examples
Architect	Requirements -> module plan, API contracts, schemas, task DAGs
Frontend	UI issues/features -> patches + browser/test evidence
Backend	API/service issues -> patches + unit/integration tests
Database	Schema/query/index tasks -> migrations + correctness tests
Debugger	Compiler/test/runtime failure -> diagnosis -> minimal repair
Reviewer	Diff + requirements -> ranked findings + concrete fix suggestions

Training stages

- **Stage 1:** clean SFT on verified role examples.
- **Stage 2:** trajectory SFT using search/read/edit/test interactions.
- **Stage 3:** preference training using passing vs failing patches and efficient vs wasteful trajectories.
- **Stage 4:** optional execution-grounded RL using build/test success as reward.
- **Stage 5:** distill manager behavior from stronger agent trajectories.

Do not over-specialize early. Keep a meaningful amount of general coding data in every worker so the frontend model does not become spiritually incapable of reading SQL.

7. Teacher Data Factory

A stronger teacher model creates candidate plans, patches, tests, reviews, and complete tool trajectories. The crucial step is verification: teacher output becomes training data only after objective checks or strong filtering.

```
Task source | +--> real OSS issue +--> synthetic feature +--> injected bug +--> compiler mutation +-->
curated web-app task | v Teacher agent | N candidate traces | v build / test / lint / browser verifier |
+----+----+ | | pass fail | | positive negative/preference data | dedup + quality scoring | role-labelled
training corpus
```

Store repository commit, task, role, tool trace, patch, tests, execution logs, teacher/model version, token cost, result, and deduplication hash for every example.

8. Quantization and Adapter Deployment

Variant	Purpose
BF16	Training/reference quality ceiling
Q8	High-fidelity deployment reference
Q6/Q5	Likely best quality-memory compromise
Q4	Primary constrained-VRAM target

For every quantization level, rerun the same held-out coding and agent benchmarks. Coding can degrade in ways that generic perplexity does not reveal, especially multi-step debugging and tool-call formatting.

Adapter deployment experiments

- Base + one LoRA loaded at a time.
- Adapter hot-swap latency.
- Merged adapter checkpoint versus dynamic adapter application.
- One multi-role adapter versus six separate adapters.
- Role token/prompt only versus actual adapter specialization.

9. Orchestrator Implementation

The orchestrator is a state machine around model inference, tools, verification, and context management.

```
while not done: state = load_project_state() next_action = route(state) if next_action.kind == "worker":
context = retrieve_minimal_context(state, next_action.role) result = run_worker(next_action.role, context)
validate_tool_calls(result) execute_actions_in_sandbox(result) update_state() elif next_action.kind ==
"verify": run_build_tests_browser_checks() update_state() elif next_action.kind == "review":
run_reviewer_on_diff() update_state() enforce_budgets() detect_loops() checkpoint_state()
```

Context management

- Use symbol/code search before file reads.
- Chunk large files and retrieve relevant ranges.
- Keep an architecture/project summary updated after major changes.
- Pass test failures plus implicated code, not the entire repository.
- Maintain a structured project state outside the LLM context window.

10. Custom Inference Runtime Integration

Your custom runtime can become the deployment layer once the agent works with a reference engine. Do not combine model-training bugs, orchestration bugs, and hand-written inference-kernel bugs into one debugging session unless you have developed a personal dislike of weekends.

Runtime milestone	Capability
1	Load quantized base weights and metadata
2	Tokenizer parity
3	Transformer forward pass parity
4	KV-cached autoregressive decoding
5	Q8 then Q4/Q5 quantized matmul
6	Adapter/LoRA application
7	CPU/GPU offload controls
8	Prompt/prefix cache
9	Adapter hot swapping
10	Profiling: TTFT, decode tok/s, memory, swap latency

Keep a reference runtime available. Every optimization should be checked against known logits/output on deterministic prompts.

11. Evaluation and Research Protocol

The strongest research question is not whether six agents sound clever. It is whether specialization and orchestration produce more correct software per unit of compute/time than a single general model.

System	Compare
Base 8B alone	No adapters, same tools
Specialized 8B alone	Best single adapter
Multi-agent system	Manager + role adapters + tools
Multi-agent without learned manager	Deterministic routing
Multi-agent without adapters	Role prompts only
Commercial/general baseline	Same tasks and approximately comparable tool budget

Metrics

- Feature/task completion rate.
- Repository issues resolved.
- Hidden tests passed.
- Build success rate.
- Regression rate.
- Average worker/model calls per solved task.
- Tool calls and wall-clock time.
- Tokens generated per solved task.
- Peak VRAM/RAM.
- Adapter-switch latency.
- Time to first token and decode tokens/s.
- Cost per solved task for hosted teacher/baseline.

Ablations

- One general model vs specialists.
- Role prompting vs LoRA adapters.
- LLM manager vs deterministic manager vs hybrid.
- No reviewer vs reviewer.
- No browser verification vs browser verification.
- Q8/Q6/Q5/Q4.
- Full context vs retrieval.
- Answer-only distillation vs full tool-trajectory distillation.

12. Repository Layout

```
local-coding-swarm/ ■■■ manager/ ■ ■■■ router/ ■ ■■■ planner/ ■ ■■■ state/ ■■■ workers/ ■ ■■■
architect/ ■ ■■■ frontend/ ■ ■■■ backend/ ■ ■■■ database/ ■ ■■■ debugger/ ■ ■■■ reviewer/ ■■■
adapters/ ■■■ agent/ ■ ■■■ tools/ ■ ■■■ context/ ■ ■■■ sandbox/ ■■■ teacher/ ■ ■■■ generation/ ■ ■■■
verification/ ■■■ training/ ■ ■■■ sft/ ■ ■■■ preference/ ■ ■■■ configs/ ■■■ runtime/ ■ ■■■ tokenizer/
■ ■■■ tensor/ ■ ■■■ model/ ■ ■■■ quant/ ■ ■■■ lora/ ■■■ eval/ ■ ■■■ tasks/ ■ ■■■ harness/ ■ ■■■
metrics/ ■ ■■■ reports/ ■■■ demo/ ■■■ docs/
```

13. Implementation Roadmap

Weeks	Deliverable
1-2	Define benchmark, choose base, build sandbox/tool API, baseline evaluation.
3-4	Build deterministic manager + one general coding worker; complete end-to-end repo repair loop.
5-6	Teacher-data pipeline and verified role-labelled dataset.
7-8	Train backend + debugger adapters; prove adapter switching and measurable gains.
9-10	Train remaining architect/frontend/database/reviewer adapters.
11-12	Hybrid manager; task DAG, structured project state, retrieval/context compression.
13-14	Autonomous Next.js/NestJS/Postgres application benchmark.
15	Quantization sweep and 6-8 GB VRAM optimization.
16-18	Integrate custom inference runtime: tokenizer, forward pass, KV cache, quantized decode.
19	LoRA support/hot swap in custom runtime; profiling.
20	Full ablations, commercial/general baseline comparison, failure analysis.
21-22	Demo UI/CLI, technical report, model/dataset cards, reproducibility cleanup.

A solid portfolio version can therefore be targeted at roughly **5-6 months part-time**. Training every component from scratch before proving the orchestrator would stretch this unnecessarily.

14. Hardware and Compute Plan

Resource	Recommendation
Local GPU	6 GB workable for aggressive Q4 prototype; 8 GB preferred
System RAM	32 GB minimum recommendation; 64 GB useful for experimentation/offload
Storage	Fast SSD; model checkpoints, datasets, containers, and logs can consume hundreds of GB
Training	Rent higher-memory GPUs for adapter/SFT experiments rather than forcing training onto the deployment GPU
CPU	Useful for manager, repo indexing, sandbox workloads, and partial offload
Deployment	One active worker at a time; keep orchestration and tools concurrent on CPU

The local machine is the **deployment target**, not necessarily the training machine. That distinction saves enormous amounts of pointless suffering.

15. Failure Modes

Failure	Mitigation
Manager loops between workers	State-machine guards, retry limits, loop detection
Workers overwrite each other	Git checkpoints, task ownership, minimal patches
Small model misses repo context	Search/retrieval, symbol index, architecture summary
Specialist loses general ability	Mix general coding examples into adapter training
Reviewer nitpicks instead of finding bugs	Train/rank on verified defects and requirement violations
Q4 damages reasoning	Use Q5/Q6 where memory permits; quantify regression
Browser agent is brittle	Prefer semantic DOM/accessibility data; screenshots as secondary evidence
Generated command is dangerous	Sandbox, allowlists, no host secrets
Teacher data contains errors	Execution verification and multi-candidate filtering
Benchmark looks suspiciously amazing	Leakage checks, private held-out repos, publish full protocol

16. Definition of Done

Deliverable	Completion criterion
Manager	Can decompose a feature into a dependency graph and recover from failed subtasks.
Workers	At least 5 specialized roles show measurable gains or efficiency benefits over role prompting alone.
Agent tools	Can search, edit, build, test, inspect browser output, and produce git diffs safely.
Web-app benchmark	System builds a non-trivial full-stack application from specification and passes predefined acceptance tests.
Repair benchmark	System resolves held-out repository bugs under fixed budgets.
Quantized deployment	Useful configuration runs on target 6-8 GB VRAM with documented RAM/context limits.
Custom runtime	Produces validated inference and supports the deployment quantization; LoRA support is strongly preferred.
Evaluation	Full baseline, ablation, memory, latency, throughput, correctness, and failure report.
Reproducibility	Another engineer can run the demo/evaluation from documented setup.
Portfolio	Architecture diagram, demo video, technical report, benchmark tables, model/data cards.

Final project claim	Build toward a defensible statement such as: 'A local manager + specialist-adapter coding system running a shared quantized 8B base completed more held-out full-stack engineering tasks than the same base used as a single general agent, while fitting within a consumer 6-8 GB VRAM deployment envelope.'

If it also beats a commercial small-model baseline under the same task/tool protocol, report that separately with the exact model version, date, limits, and complete test set. Narrow evidence beats grandiose benchmark theater.